

Maxwell_Kokkos

Documentation for the Kokkos port of the SBP–SAT Maxwell solver

HPC course — SMS 751/2255

April 2026

Contents

1	Overview	3
2	The continuum system	4
2.1	The flux form	4
3	Spatial discretization	5
3.1	Grid, decomposition, and indexing	5
3.2	Summation-by-parts (SBP) operators	5
3.3	Optional Kreiss–Oliger dissipation	6
4	Time integration	6
5	Boundary conditions: the SBP-SAT penalty method	6
5.1	Why weak imposition	6
5.2	Characteristic decomposition	7
5.3	The penalty form	7
5.4	Application to Maxwell’s equations	8
5.5	Edges and corners: additivity	9
5.6	Stability, τ , and CFL at corners	9
6	Source models	9
6.1	Plane wave source	9
6.2	Paraxial Gaussian beam	10
6.3	Smooth turn-on in time	10
6.4	TE waveguide mode	11
7	Material model	11
8	Limitations	11
8.1	Transverse-face reflections	11
8.2	Paraxial approximation	12
8.3	Resolution / containment / paraxial trade-off	12
8.4	CFL at corners	12

8.5	Plane wave with six-face SAT	12
8.6	Angular (oblique) reflections from every SAT face	12
9	Perfectly Matched Layers: a future extension	13
9.1	The mathematical mechanism: complex coordinate stretching	13
9.2	From frequency domain to time domain	14
9.3	The damping profile and what a PML would add	14
10	Software architecture: Kokkos integration	14
10.1	Driving design choice	15
10.2	Three-region kernel partition	15
10.3	Ghost-zone exchange	15
10.4	Reductions	16
10.5	File map	16
11	Build	16
11.1	Prerequisites	16
11.2	Default OpenMP build	17
11.3	CUDA build	17
11.4	CUDA-aware MPI	17
11.5	CMake options reference	17
11.6	Source-level switches	17
12	Running	17
12.1	Parameter file structure	18
12.2	Two canonical configurations	19
12.3	Output layout	19
12.4	Visualization	20
12.5	Checkpoint restart	20
12.6	Diagnostics and common problems	20
13	Tests	21
13.1	<code>test_topology</code>	21
13.2	<code>test_rk4</code>	21
13.3	<code>test_sync</code>	21
13.4	Running the suite	21
14	Verification: convergence testing	22
14.1	Test problem	22
14.2	The four configurations	22
14.3	Why the expected rates are 4 and 3	23
14.4	Running the convergence test	23
14.5	Baseline results (Kokkos OpenMP backend, validated)	23
14.6	Interpreting a failed test	24
15	Validation status	25
15.1	Verified on the OpenMP backend	25
15.2	Built but not yet runtime-verified	25
15.3	Recommended GPU-side test sequence	25

16 Known issues and toolchain workarounds	25
16.1 OpenMPI 4.x C++ bindings vs. <code>nvcc</code>	25
16.2 GCC 12 + <code>toml++</code> + C++20	26
16.3 Single-GPU multi-rank	26
16.4 C++20 hard requirement	26
A Summary of SAT formulas	27

Abstract

`Maxwell_Kokkos` is a three-dimensional finite-difference solver for Maxwell’s equations in a linear (possibly inhomogeneous) dielectric, coupled to a hyperbolic constraint-damping system. The spatial discretization uses fourth-order summation-by-parts (SBP-4-2) operators with one-sided closures at every physical face. Boundary conditions are imposed weakly via Simultaneous Approximation Term (SAT) penalties on all six Cartesian faces. The time integrator is classical fourth-order Runge–Kutta. The code is the Kokkos port of the Week-11 `Maxwell_Penalty` reference: the physics and the discretization are unchanged, but every compute kernel is expressed as a `Kokkos::parallel_for` or `parallel_reduce` so the same source builds for OpenMP on a CPU host and for CUDA on a single GPU. This document covers the physics, the numerical scheme, the SBP-SAT penalty derivation, source models and their limitations, the Kokkos integration, build and run instructions, tests, validation status, and the appendix with explicit per-face SAT formulas.

1 Overview

`Maxwell_Kokkos` is a Kokkos re-implementation of `Maxwell_Penalty` from Week 11. The physics, the discretization, and the user-facing interface (TOML configuration, HDF5 output, MPI Cartesian decomposition) are unchanged from the parent code; the compute kernels have been ported to `Kokkos::parallel_for` and `Kokkos::parallel_reduce` so the same source builds for OpenMP on a CPU host and for CUDA on a single GPU. Companion documents:

- `../Week11/Maxwell_Penalty/CLAUDE.md` — parent code notes (TOML schema, source types, lens demo, convergence rates, SBP–SAT sign convention warnings).
- `../Week11/Maxwell_Penalty/doc/documentation.tex` — the parent document this one is partially derived from. Many of the physics, characteristic, and limitations sections are imported verbatim from there.
- `../Week11/Maxwell/Develop_Penalty.md` — continuum derivation of the SBP–SAT penalty for the extended Maxwell system.
- `GPU_Maxwell.md` — the implementation plan that produced this code.
- `Report.md` — focused report on the include and build-option design as actually shipped.

The remainder of this document is organised as follows. Sections 2–7 cover the physics: the extended-Maxwell system, the spatial discretization, the SBP–SAT penalty, the available analytic sources, and the material model. Section 8 catalogues the limitations of characteristic-based absorbing boundaries and Section 9 sketches the Perfectly Matched Layer extension that would address them. Section 10 is the new value-add of this port: how the Kokkos integration is laid out, the design choices that let the existing SymPy-generated SAT macros work unchanged, and the file map. Sections 11–12 cover building and running the code, including the full TOML parameter

schema and the two canonical configurations. Section 13 documents the test suite, Section 14 the convergence study (the strongest correctness check), Section 15 the overall validation status, and Section 16 the toolchain pitfalls discovered when bringing the code up on an A100 host. Appendix A gives the explicit per-face SAT formulas.

2 The continuum system

The code evolves the electromagnetic field $(\mathbf{E}, \mathbf{B})$ in a linear, possibly inhomogeneous, dielectric. Let $\varepsilon(\mathbf{x})$ be the permittivity, $\mu(\mathbf{x})$ the permeability, and $\sigma(\mathbf{x})$ the conductivity (Ohmic damping). Define $\mathbf{D} = \varepsilon \mathbf{E}$ and $\mathbf{H} = \mathbf{B}/\mu$. In Gaussian units, Maxwell’s equations are

$$\partial_t \mathbf{D} = \nabla \times \mathbf{H} - 4\pi \mathbf{J}, \quad (1)$$

$$\partial_t \mathbf{B} = -\nabla \times \mathbf{E}, \quad (2)$$

$$\nabla \cdot \mathbf{D} = 4\pi \rho, \quad (3)$$

$$\nabla \cdot \mathbf{B} = 0, \quad (4)$$

with Ohm’s law $\mathbf{J} = \sigma \mathbf{E}$. Equations (1)–(2) are the time-evolution (*hyperbolic*) equations; (3)–(4) are the divergence constraints, preserved by the time evolution in the continuum.

Numerically this preservation is not automatic. Round-off and truncation error in the discretized curl operators can seed small constraint violations that, in a naive scheme, grow without bound. The standard remedy — following Munz et al. — is to extend the system with two auxiliary scalar fields Ψ_D, Ψ_B that act as Lagrange multipliers for the constraints:

$$\partial_t \mathbf{D} - c^2 \nabla \Psi_D = \nabla \times \mathbf{H} - 4\pi \mathbf{J}, \quad (5)$$

$$\partial_t \mathbf{B} - c^2 \nabla \Psi_B = -\nabla \times \mathbf{E}, \quad (6)$$

$$\partial_t \Psi_D = \nabla \cdot \mathbf{D} - 4\pi \rho - \kappa_D \Psi_D, \quad (7)$$

$$\partial_t \Psi_B = \nabla \cdot \mathbf{B} - \kappa_B \Psi_B. \quad (8)$$

At $\Psi_D = \Psi_B = 0$ the original Maxwell system is recovered. For $\kappa_{D,B} > 0$ the Ψ fields satisfy damped wave equations whose source is the constraint violation: any residual violation radiates outward at speed c and is exponentially damped on timescale $1/\kappa$. This absorbs round-off and truncation noise, and absorbs the $O(1/(kw_0)^2)$ constraint residual of the paraxial beam source (§6.2).

The code evolves nine fields, $U = (D_x, D_y, D_z, B_x, B_y, B_z, \Psi_D, \Psi_B, \rho)^\top$, plus five auxiliaries $(1/\varepsilon, 1/\mu, \sigma, c_D, c_B)$ where the last two are diagnostic constraint values. The charge density ρ is carried as an evolved variable for generality, driven by $\partial_t \rho + \nabla \cdot \mathbf{J} = 0$; for the simulations shipped with the code, ρ and $\mathbf{J}$ are initially zero and remain zero up to round-off. Slot ordering is fixed in the `evolved_slot` and `auxilliary_slot` enums in `maxwell_eqs.hpp`; the C-version’s discipline of “one source of truth for variable order” is preserved verbatim.

The material parameters ε, μ, σ are stored in inverse form `ieps` = $1/\varepsilon$, `imu` = $1/\mu$, `sigma` = σ as auxiliary gridfunctions. They may vary in space; the default beam-through-lens demo places an elliptical-contour permittivity bump (§7).

2.1 The flux form

The spatial derivatives in the extended system can be written as a quasi-linear hyperbolic system

$$\partial_t U + A_x \partial_x U + A_y \partial_y U + A_z \partial_z U = S(U, \mathbf{x}), \quad (9)$$

where S collects source terms (conductivity damping, constraint relaxation, material-gradient couplings) and the three flux matrices A_x, A_y, A_z encode spatial propagation. Their characteristic decomposition is what the SAT penalty needs; we return to it in §5.

3 Spatial discretization

3.1 Grid, decomposition, and indexing

The domain is a rectangular parallelepiped $[x_0, x_n] \times [y_0, y_n] \times [z_0, z_n]$, discretized on a uniform Cartesian grid of global size (N_x, N_y, N_z) with spacings dx, dy, dz . The grid is distributed across MPI ranks via a Cartesian topology chosen automatically by greedy prime factorization. Each rank owns a contiguous block surrounded by ghost zones of width g_s (default 3). For physical boundaries the ghost zones are absent and the rank stores real data at every index; for MPI-internal and periodic boundaries, the ghost zones are synchronised via non-blocking point-to-point communication between RK stages.

The local index into a 3D field f at grid point (i, j, k) is $\mathbf{ijk} = i + jN_x + kN_xN_y$, with i the fastest-varying index. Strides are $d_i = 1$, $d_j = N_x$, $d_k = N_xN_y$. Every finite-difference stencil takes a stride argument so it can operate along any axis. The Kokkos port keeps this offset arithmetic identical: `Kokkos::View<double***, LayoutLeft>` of shape (N_x, N_y, N_z) has the same flat-index offset, so the existing stencils and SAT macros work unchanged — see §10.

3.2 Summation-by-parts (SBP) operators

The spatial derivatives are computed by summation-by-parts finite differences of order $(4, 2)$ — fourth-order in the interior, second-order one-sided closures at the first four rows on each boundary. A $(4, 2)$ SBP operator D approximating d/dx on a grid of N points factorizes as

$$D = H^{-1}Q, \quad Q + Q^\top = B = \text{diag}(-1, 0, \dots, 0, +1), \quad (10)$$

where H is a positive-definite diagonal weight matrix (the “norm”) and Q is almost antisymmetric, the asymmetry captured entirely by the boundary term B . In the $(4, 2)$ case

$$H = h \cdot \text{diag}(17/48, 59/48, 43/48, 49/48, 1, 1, \dots). \quad (11)$$

The key stability-enabling identity is

$$\langle u, Dv \rangle_H + \langle Du, v \rangle_H = u_N v_N - u_0 v_0, \quad (12)$$

where $\langle u, v \rangle_H = \sum_i H_{ii} u_i v_i$. This is the discrete analogue of integration by parts on a finite interval — the boundary terms appear exactly, without truncation error, and this identity is what lets us prove a discrete energy estimate.

The interior stencil (rows $i = 4, \dots, N - 5$) is the standard fourth-order centered difference

$$(Du)_i = \frac{u_{i-2} - 8u_{i-1} + 8u_{i+1} - u_{i+2}}{12h} + O(h^4).$$

The four near-boundary rows use one-sided Strand (1994) closures; at the lower boundary,

$$\begin{aligned}(Du)_0 &= \frac{-24u_0 + 59u_1 - 8u_2 - 3u_3}{17h}, \\(Du)_1 &= \frac{-u_0 + u_2}{2h}, \\(Du)_2 &= \frac{4u_0 - 59u_1 + 59u_3 - 4u_4}{86h}, \\(Du)_3 &= \frac{3u_0 - 59u_2 + 64u_4 - 8u_5}{98h},\end{aligned}$$

and symmetric closures at the upper boundary. These are implemented as `SBP42_L0`–`SBP42_L3` and the upper-boundary counterparts `SBP42_RN`–`SBP42_RNm3` in `derivatives.hpp`. In the Kokkos port each stencil function is annotated `KOKKOS_INLINE_FUNCTION` so it can be called from device kernels.

3.3 Optional Kreiss–Oliger dissipation

When `[solver].use_dissipation = true` the RHS is augmented by a 5th-order Kreiss–Oliger dissipation operator

$$\dot{u}_i += -\frac{\sigma_{\text{KO}}}{\Delta x} \cdot \frac{1}{64} (u_{i-3} - 6u_{i-2} + 15u_{i-1} - 20u_i + 15u_{i+1} - 6u_{i+2} + u_{i+3}),$$

applied separately along x , y , z . The operator is skipped on the outer three rows of each axis. Useful for damping grid-scale noise from oblique partial reflections; see §8.6.

4 Time integration

Time integration is classical explicit fourth-order Runge–Kutta:

$$\begin{aligned}k_1 &= f(t_n, u_n), & u_n^{(1)} &= u_n + \frac{\Delta t}{2} k_1, \\k_2 &= f(t_n + \frac{\Delta t}{2}, u_n^{(1)}), & u_n^{(2)} &= u_n + \frac{\Delta t}{2} k_2, \\k_3 &= f(t_n + \frac{\Delta t}{2}, u_n^{(2)}), & u_n^{(3)} &= u_n + \Delta t k_3, \\k_4 &= f(t_n + \Delta t, u_n^{(3)}), & u_{n+1} &= u_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4).\end{aligned}$$

The timestep is chosen by the CFL condition $\Delta t = c \cdot \min(\Delta x, \Delta y, \Delta z)$ with the safety factor c taken from the TOML (`[solver].cfl_factor`). For the pure SBP interior (no SAT), stability permits $c \lesssim 1$. With SAT penalties on all six faces, the effective spectral radius increases near corners where multiple penalties stack, and the usable CFL drops to $c \approx 0.2$ in practice (§5.6).

In the Kokkos port the C version’s pointer-rebinding trick (where each `vars[v]->dot` pointer was reseated between RK stages to alias `K1 . . K4`) is replaced by an explicit integer `kidx` parameter to the time-derivative function. `DECLARE_EVOLVED_VARS` uses `kidx` to pick the correct `K[kidx].data()` pointer. This keeps the kernel body identical to the C version while removing a non-obvious dataflow.

5 Boundary conditions: the SBP-SAT penalty method

5.1 Why weak imposition

The naïve approach to boundary conditions — “overwrite the evolution at boundary nodes with the prescribed boundary value” — is called *strong imposition*. It is simple and often works, but it breaks

the SBP energy estimate: a boundary row whose equation is replaced loses the clean integration-by-parts structure, and stability must be argued separately, usually by numerical experiment.

The Simultaneous Approximation Term (SAT) method imposes boundary conditions *weakly*, as an additive penalty. The PDE is retained at every row, including the boundary row, and a correction is *added* that pulls the solution toward the prescribed value. The SAT term is tuned so the full semi-discrete system admits a discrete energy estimate: summing the equations against $U^\top H$ and using the SBP identity produces boundary terms that are exactly cancelled by the SAT penalty. This gives provable stability for any $\tau \geq 1/2$ (§5.6), independent of the particular PDE coefficients or grid spacing.

Property	Strong imposition	SBP-SAT (weak)
Boundary equation	Replaced entirely	Interior RHS + penalty
Energy estimate	No formal proof	Yes, provable for $\tau \geq 1/2$
Stability w/ non-uniform ε, μ	No guarantee	Provable
Convergence order	Same as interior closures	Same as interior closures
Penalty scaling	N/A	$\tau \cdot H_{00}^{-1}/h$

Both methods give the same formal convergence order — the boundary truncation error is set by the SBP one-sided closure, not by how the boundary condition is imposed. The pedagogical reason to prefer SBP-SAT is the discrete energy estimate, which extends to variable coefficients, multi-dimensional grids, and characteristic couplings.

5.2 Characteristic decomposition

The penalty acts on the *incoming* characteristic at each face. For a hyperbolic system $\partial_t U + A \partial_x U = 0$ with flux matrix A , diagonalize

$$A = V \Lambda V^{-1}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n),$$

and split into positive and negative eigenvalue parts:

$$A^+ = V \Lambda^+ V^{-1}, \quad A^- = V \Lambda^- V^{-1}, \quad \Lambda^\pm = \text{diag}(\max(\pm \lambda_i, 0)).$$

The characteristic variables $W = V^{-1}U$ satisfy $(\partial_t + \lambda_i \partial_x)W_i = 0$; modes with $\lambda_i > 0$ propagate in $+x$ (incoming at the lower face), modes with $\lambda_i < 0$ propagate in $-x$ (incoming at the upper face).

At the lower- x face, we prescribe the incoming characteristics (the $+\lambda$ modes); the $-\lambda$ modes are outflow and are determined by the interior dynamics. At the upper- x face the roles are reversed.

5.3 The penalty form

At the lower- x boundary (grid index $i = 0$), the SAT correction is

$$\dot{U}_0 = \tau \cdot \frac{(H^{-1})_{00}}{h} \cdot A^+ \cdot (U_0 - g), \quad (13)$$

where g is the prescribed incoming state and $\tau \geq 1/2$ is the penalty strength. The code uses $\tau = 1$ by default; $(H^{-1})_{00} = 48/17$ for the $(4, 2)$ SBP norm. The stencil operates only on the single boundary row — rows $i = 1, 2, 3$ receive the SBP one-sided derivative (§3.2) but *no* additional SAT term; they contribute to the energy argument through the norm integral. The analogous correction at the upper boundary uses A^- (or equivalently the positive-definite projection $|A^-|$) and the incoming data for that face:

$$\dot{U}_{N-1} = \tau \cdot \frac{(H^{-1})_{N-1,N-1}}{h} \cdot |A^-| \cdot (U_{N-1} - g). \quad (14)$$

By symmetry $(H^{-1})_{N-1,N-1} = (H^{-1})_{00} = 48/17$.

5.4 Application to Maxwell's equations

For the extended Maxwell system, the z -axis flux matrix decomposes into four independent 2×2 blocks plus one trivial 1×1 block:

Block	Variables	Eigenvalues	Incoming char. at lower z (w^+)
EM-1	(B_x, D_y)	$\pm c$	$B_x - PP \cdot D_y$
EM-2	(B_y, D_x)	$\pm c$	$B_y + PP \cdot D_x$
Div- B	(B_z, Ψ_B)	± 1	$B_z - \Psi_B$
Div- D	(D_z, Ψ_D)	± 1	$D_z - \Psi_D$
Charge	ρ	0	(none)

Here $c = 1/\sqrt{\varepsilon\mu}$ is the local light speed ($\sqrt{\mathbf{i}\mathbf{eps} \cdot \mathbf{i}\mathbf{mu}}$ in the code's normalisation) and $PP = \sqrt{\varepsilon/\mu}^{-1} = \sqrt{\mathbf{i}\mathbf{eps}/\mathbf{i}\mathbf{mu}}$ is the impedance ratio. The divergence blocks have eigenvalues ± 1 because constraint violations propagate at the normalised speed set by the extended system, independent of material. The w^+ column lists, for each block, the left-eigenvector combination that is the incoming characteristic at the lower- z face; the upper- z face injects the opposite combination (w^-) with the signs of the cross-coupling terms reversed.

For the EM-1 block (B_x, D_y) , the flux matrix is

$$A_z^{(\text{EM1})} = \begin{pmatrix} 0 & -\mathbf{i}\mathbf{eps} \\ -\mathbf{i}\mathbf{mu} & 0 \end{pmatrix}.$$

The right eigenvector for $\lambda = +c$ is $v^+ = (1, -1/PP)^\top$; the left eigenvector ($L^+A = cL^+$) is $L^+ = (1, -PP)^\top$; the positive projection is

$$A^+ = \frac{c}{L^+ \cdot v^+} v^+(L^+)^\top = \frac{c}{2} \begin{pmatrix} 1 & -PP \\ -1/PP & 1 \end{pmatrix}.$$

Applying to the deviation $\delta U = (B_x - g_{B_x}, D_y - g_{D_y})$:

$$\begin{aligned} (A^+ \delta U)_{B_x} &= \frac{c}{2} (\delta B_x - PP \cdot \delta D_y), \\ (A^+ \delta U)_{D_y} &= \frac{c}{2} (-\delta B_x / PP + \delta D_y). \end{aligned}$$

The incoming characteristic for this block is therefore $w^+ = L^+ \cdot U = B_x - PP \cdot D_y$.

Sanity check on a known solution. For a y -polarised plane wave travelling in $+z$ (incoming at the lower- z boundary), the fields satisfy $D_y = \psi$, $B_x = -\psi$ (the sign makes $\mathbf{E} \times \mathbf{B}$ point in $+\hat{z}$), so

$$w^+ = -\psi - PP \cdot \psi = -(1 + PP)\psi \neq 0$$

as expected for an incoming mode. A sign error that flipped any coupling term ($+PP$ instead of $-PP$, say) would give $w^+ = 0$ on this same physical solution, and the SAT would silently fail to inject the wave. The hand-written sign convention in `Maxwell/src/PenaltyImplementation.md` contains errors of exactly this form; the SymPy-emitted macros in `simple_maxwell.h` are the authoritative version, derived from scratch by SymPy diagonalization of M_a and verified against the traveling-wave check above.

The implementation lives in `simple_maxwell.h` as six preprocessor macros: `APPLY_SAT_LOWER_{X,Y,Z}` and `APPLY_SAT_UPPER_{X,Y,Z}`. Only the lower- z face has nonzero g in the plane-wave and Gaussian-beam configurations (the incoming source field, §6); the other five faces use $g = 0$, which corresponds to a perfectly absorbing boundary for waves leaving normally. See §8.6 for the angular caveat.

5.5 Edges and corners: additivity

At an edge (two physical faces meet) and at a corner (three faces meet), the SAT penalties are *additive*: each face contributes its own penalty term independently. No cross-term or corner-specific correction is needed. This additivity is a direct consequence of the tensor-product structure: the 3D discrete energy norm $H = H_x \otimes H_y \otimes H_z$ decomposes the boundary contributions into independent surface integrals over each face, each cancelled by its own SAT term.

Concretely, at the corner $(i, j, k) = (0, 0, 0)$ with all three axes physical, the RHS receives

$$\dot{U}_{0,0,0} = \tau \left[\frac{H_0^{-1}}{dx} A_x^+ \delta U + \frac{H_0^{-1}}{dy} A_y^+ \delta U + \frac{H_0^{-1}}{dz} A_z^+ \delta U \right], \quad (15)$$

where the three projections act on the same state δU but with axis-specific flux matrices. This is the only corner-aware term in the code — the SBP derivatives at the corner use one-sided closures in each direction independently, with no other correction.

5.6 Stability, τ , and CFL at corners

The standard SBP-SAT stability theorem (Svärd & Nordström) requires $\tau \geq 1/2$ for a single-face penalty on a scalar hyperbolic equation. For a system with n characteristic speeds on a tensor-product multidimensional grid, the same bound applies per face. The code uses $\tau = 1$ as a comfortable margin.

At corners, three SAT terms with per-face scaling $\tau(H_0^{-1})/h$ stack additively on the same boundary row. The effective spectral radius of the discrete operator grows at the corner, and the usable CFL number drops. Empirically, the six-face-physical scheme is stable at $\text{cfl} \approx 0.2$ with classical RK4 — tighter than $\text{cfl} \approx 0.5$ safe for an interior-only (or two-face) scheme. Raising τ beyond 1 tightens the CFL further; reducing τ toward $1/2$ relaxes it at the cost of stability margin.

6 Source models

The lower- z face (or, for the convergence-test source, every physical face) has nonzero boundary data g corresponding to a specified incoming electromagnetic field. Three sources are implemented:

- **plane_wave**: a bump-enveloped plane wave travelling in $+\hat{z}$ (§6.1). Pair with periodic x, y for the formal convergence reference (Option B).
- **gaussian_beam**: a paraxial collimated beam injected at the lower- z face (§6.2). Used with all faces non-periodic for the headline beam-through-lens demonstration (Option A).
- **te_waveguide_mode**: an exact vacuum Maxwell TE mode of a rectangular unit-cube waveguide (§6.4), used as SAT data on every physical face for the four-configuration convergence study (§14).

Selection is via `[source].type` in the TOML. Per-source parameters live in dedicated sub-tables; the driver parses all three so switching sources is a one-line edit.

6.1 Plane wave source

A transverse electromagnetic wave travelling in $+\hat{z}$, linearly polarised as $\mathbf{E} = (E_0^x \hat{x} + E_0^y \hat{y}) f(t - z) \cos(k(z - t))$, modulated by a C^2 smooth bump envelope f that turns on over an interval $[a, b]$. This satisfies the vacuum Maxwell equations exactly (independent of k), so in a configuration with

x, y periodic and z non-periodic this is an *exact* reference for formal convergence testing of the SBP-SAT scheme.

The plane wave has no transverse structure: the field at $x = 0$ and $x = L_x$ is identical. Combining this source with non-periodic x, y would put a full-amplitude wavefield at the transverse walls, where the $g = 0$ SAT pulls toward zero and excites a numerical instability. The plane wave is not usable with six-face SAT (§8).

6.2 Paraxial Gaussian beam

Models a collimated laser beam entering at the lower- z face, optionally converging toward a waist at $z = z_w$. The paraxial scalar amplitude is

$$\psi(\mathbf{x}, t) = E_0 \cdot \frac{w_0}{w(z)} \exp\left(-\frac{r^2}{w(z)^2}\right) \cos\left(kz - \omega t + \frac{kr^2}{2R(z)} - \varphi_G(z)\right), \quad (16)$$

with $r^2 = x^2 + y^2$, $\omega = kc$, and

$$\begin{aligned} z_R &= \frac{1}{2}kw_0^2 \quad (\text{Rayleigh range}) \\ w(z) &= w_0\sqrt{1 + ((z - z_w)/z_R)^2} \\ 1/R(z) &= (z - z_w)/((z - z_w)^2 + z_R^2) \\ \varphi_G(z) &= \arctan((z - z_w)/z_R) \quad (\text{Gouy phase}). \end{aligned}$$

The implementation in `analytic_solutions.hpp` uses an electric Hertz-vector construction $\mathbf{\Pi} = \hat{x} \text{Re}[\Phi_c]$ with $\Phi_c = (E_0/k^2) A(r, z) e^{i\gamma}$ and sets $\mathbf{D} = \nabla \times \nabla \times \mathbf{\Pi}$, $\mathbf{B} = \partial_t(\nabla \times \mathbf{\Pi})$. This makes $\nabla \cdot \mathbf{D} = \nabla \cdot \mathbf{B} = 0$ *exactly* by $\nabla \cdot (\nabla \times \cdot) \equiv 0$, so the constraint-damping fields stay at numerical-noise level even though Φ only solves the wave equation to $O((kw_0)^{-2})$.

Convergence vs. divergence. The sign of $1/R(z)$ determines the wavefront curvature. For $z < z_w$, $R < 0$: the wavefronts are concave toward the beam axis and the beam converges. For $z > z_w$, $R > 0$ and the beam diverges. Placing z_w in the middle of the domain produces a beam that converges through the first half, reaches minimum spot at the middle, then diverges.

Collimation. A beam is “nearly collimated” (laser-like) over a propagation distance L when $z_R \gg L$. The spot size varies as $w(L)/w_0 = \sqrt{1 + (L/z_R)^2}$, so $z_R = 10L$ gives 0.5% broadening, and $z_R = 3L$ gives 5%. Fixed domain L then requires large kw_0^2 , which costs transverse resolution (§8).

Paraxial residual. (16) satisfies the full Maxwell equations only to $O(1/(kw_0)^2)$. The residual manifests as small but nonzero $\nabla \cdot \mathbf{E}$ and $\nabla \cdot \mathbf{B}$ when the beam is injected; the constraint-damping fields Ψ_D, Ψ_B absorb and radiate it on timescale $1/\kappa_{D,B}$. $kw_0 \gtrsim 5$ keeps the residual below 4%, well within the constraint-damping system’s domain of validity.

6.3 Smooth turn-on in time

For `plane_wave`, the envelope $f(t - z)$ vanishes at $t = 0$ for all $z \geq 0$, so the initial state is identically zero and the wave enters gradually through the SAT boundary.

For `gaussian_beam`, the analytic formula (16) does not vanish at $t = 0$. Filling the interior with it would seed the domain with a state that is only a paraxial approximation; empirically this excites a numerical instability that grows exponentially regardless of CFL. To avoid it, the beam source

is multiplied by a smooth C^2 turn-on $f(t; t_a, t_b)$ that rises from 0 at $t \leq t_a$ to 1 at $t \geq t_b$. The interval is exposed as `[source.gaussian_beam].ramp_a/ramp_b`; default `[0, 1]`. The initial state is therefore identically zero, and the beam grows in through the boundary SAT as the envelope rises. Lengthening the ramp reduces paraxial-residual excitation during turn-on at the cost of delaying the onset of interesting physics.

The `te_waveguide_mode` source is an exact solution and needs no turn-on: it is non-zero at $t = 0$ and the initial state on the full grid is consistent with the boundary data at every face. The convergence test relies on this consistency (§14).

6.4 TE waveguide mode

The $\text{TE}(l, m, n)$ mode of a unit-cube rectangular waveguide propagating in $+z$, with dispersion $\omega^2 = \pi^2(l^2 + m^2 + n^2)$:

$$\begin{aligned} D_x &= \frac{\omega}{l\pi} \sin(m\pi y) \cos(l\pi x) \cos(n\pi z - \omega t), \\ D_y &= -\frac{\omega}{m\pi} \sin(l\pi x) \cos(m\pi y) \cos(n\pi z - \omega t), \\ D_z &= 0, \\ B_x &= \frac{n}{m} \sin(l\pi x) \cos(m\pi y) \cos(n\pi z - \omega t), \\ B_y &= \frac{n}{l} \sin(m\pi y) \cos(l\pi x) \cos(n\pi z - \omega t), \\ B_z &= -\frac{l^2 + m^2}{lm} \sin(n\pi z - \omega t) \cos(l\pi x) \cos(m\pi y), \end{aligned}$$

with $\Psi_D = \Psi_B = \rho = 0$. The mode numbers (l, m, n) must each be ≥ 1 (the parameter parser enforces this; division by l, m in the formula above also requires it). For even (l, m, n) the solution is also periodic on the unit cube, so the same reference works for both periodic and fully-physical boundary configurations in the convergence study.

7 Material model

Background. A uniform $(\varepsilon, \mu, \sigma)$ from `[material.background]`. The auxiliary fields $1/\varepsilon, 1/\mu, \sigma$ live in slots `IEPS_SLOT`, `IMU_SLOT`, `SIGMA_SLOT` and are constant in time.

Optional elliptical lens. If `[material].epsilon_type = "elliptical"` an elliptical-Gaussian permittivity profile is layered on top:

$$\varepsilon(x, z) = (\varepsilon_{\max} - \varepsilon_{\text{bg}}) \exp \left[-s^2 \left(\frac{(x - x_0)^2}{a^2} + \frac{(z - z_0)^2}{b^2} \right)^2 \right] + \varepsilon_{\text{bg}}, \quad (17)$$

with ε_{bg} the `epsilon` from `[material.background]`. Set once at `set_initial_data` time and held constant thereafter. Used in the Option A demo.

8 Limitations

8.1 Transverse-face reflections

The $g = 0$ SAT on the $\pm x, \pm y$ faces is correct only insofar as the physical solution has negligible amplitude there. For a Gaussian beam with $w_0 \ll L_x, L_y$ the wall tail decays as $\exp(-(L_{x,y}/2w)^2)$

and reflections sit below roundoff for $L_{x,y} \gtrsim 10 w_0$. For a plane wave (full amplitude at the walls) this boundary treatment is unsuitable — spurious reflection cascades into instability.

8.2 Paraxial approximation

The Gaussian-beam source is a paraxial approximation to Maxwell’s equations. The residual constraint violation is $O(1/(kw_0)^2)$ and is absorbed by the constraint-damping fields. This is acceptable as a physics source but means the beam is not an exact solution; the L^2 error of the numerical evolution against the paraxial reference is therefore *not* a convergence metric. Use the plane-wave (§6.1) or `te_waveguide_mode` (§6.4) source for convergence testing.

8.3 Resolution / containment / paraxial trade-off

Three constraints on the beam parameters:

- Grid resolution: $k \cdot dx \lesssim 0.6$, i.e., roughly 10 points per wavelength.
- Paraxial validity: $kw_0 \gtrsim 5$ (residual $\leq 4\%$).
- Transverse containment: $L_x/w_0 \gtrsim 5$ (preferably ≥ 10).

For a laser-like (collimated) beam, the Rayleigh range $z_R = kw_0^2/2$ must additionally be large compared with the propagation length L_z . All four together favour large k and large w_0 , raising the required N_x steeply.

8.4 CFL at corners

With all six faces treated by SAT, the scheme is stable at $\text{cfl} \approx 0.2$ with classical RK4, tighter than the $\text{cfl} \approx 0.5$ safe for the periodic- xy interior-only scheme (§5.6). Exponentially growing L^2 error usually means the CFL is too large, not a physics failure.

8.5 Plane wave with six-face SAT

The plane-wave source combined with all six SAT faces produces a documented numerical instability: the transverse walls see full-amplitude tangential field, and the $g = 0$ SAT attempts to force it to zero. This is not a bug; it is the documented failure mode of the SAT boundary for a physical configuration whose field is not transversely contained. Use periodic x, y for plane-wave runs.

8.6 Angular (oblique) reflections from every SAT face

The most important limitation of the current boundary treatment is shared by all characteristic-based absorbing boundaries: the SAT penalty is exact for waves moving *normal* to the boundary, but only partial for waves moving at oblique angles. This affects every face, not just the transverse ones:

- **Lower- z face:** the SAT’s $+c$ projection correctly leaves a head-on outgoing wave (the $-c$ mode) untouched. A wave at angle θ to the z -axis has its energy split between the $+c$ and $-c$ z -characteristics in proportions depending on $\sin \theta$; the SAT treats the $+c$ part as incoming and pulls it toward the prescribed beam data, producing a spurious reflection whose amplitude grows with θ .
- **Upper- z face:** same mechanism, with $g = 0$. A wave refracted by the lens and travelling mostly in $+z$ but at an angle is partially reflected back into the domain.

- $\pm x, \pm y$ **faces**: same mechanism again. The beam reaches these walls only as a Gaussian tail and as lens-scattered components, so the contribution is small for well-contained beams but non-negligible whenever the lens refracts energy sideways.

Concrete failure mode: beam on a high- ϵ lens. A dielectric step of permittivity $\epsilon_{\max}$ reflects a Fresnel fraction $R = ((\sqrt{\epsilon_{\max}} - 1)/(\sqrt{\epsilon_{\max}} + 1))^2$ of incident intensity at normal incidence. For $\epsilon_{\max} = 4$, $R \approx 0.11$ in intensity, $\sim 1/3$ in amplitude — a substantial portion of the beam bounces off the lens and travels back toward $-z$. The pure- $-z$ component exits cleanly; angular side-lobes generated by the elliptical edge of the lens partially reflect off the SAT faces and re-enter. After a few bounces the domain fills with low-amplitude oblique modes that interfere with the incident beam every $\lambda/2 = \pi/k$.

Mitigations within the current code.

- Shorten the simulation time so no wave completes a round trip.
- Weaken the lens ($\epsilon_{\max}$ closer to 1).
- Widen the domain so the transverse walls stay far from the beam after lens refraction.
- Enable Kreiss-Oliger dissipation (`use_dissipation = true`, `diss_coeff` ~ 0.02 – 0.1) to damp grid-scale noise from partial reflections. This does not remove the angular reflections themselves but suppresses their high-frequency signature.

None of these is a true fix; for angle-independent absorption the code would need a Perfectly Matched Layer (§9).

9 Perfectly Matched Layers: a future extension

A Perfectly Matched Layer (PML) is a finite-thickness absorbing buffer surrounding the physical domain, designed to absorb outgoing waves *at any angle and any frequency* with zero reflection at the interface (in the continuum). PML is the standard tool in production electromagnetic codes for open-domain problems. `Maxwell_Kokkos` does *not* include a PML; this section sketches what one would add.

9.1 The mathematical mechanism: complex coordinate stretching

For Maxwell in, say, an x -face PML, formally replace x -derivatives by stretched derivatives

$$\frac{\partial}{\partial x} \longrightarrow \frac{1}{s_x(x)} \frac{\partial}{\partial x}, \quad s_x(x) = 1 + \frac{\sigma_x(x)}{i\omega}, \quad (18)$$

with $\sigma_x(x) \geq 0$ the PML damping profile, zero in the physical domain and ramping up through the PML. Substituting a plane wave $e^{i(kx - \omega t)}$ in the PML region gives

$$e^{i(kx - \omega t)} \exp\left(-\int_{x_0}^x \frac{\sigma_x(x')}{c} dx'\right),$$

the same wave with an exponential envelope. Crucially, the wavenumber k and phase velocity are unchanged, so the wave does not refract at the PML interface — the match is perfect for *all* angles and *all* frequencies.

9.2 From frequency domain to time domain

The complex factor $1/s_x$ is trivial in frequency but requires care in time: dividing by $(i\omega + \sigma_x)$ is a convolution with an exponential kernel. Two standard implementations:

- **Bérenger split-field PML (1994)**: each field component is split into pieces labelled by the contributing derivative; each piece evolves under its own damping ODE. Simple but doubles the field count in the PML.
- **Convolutional PML / CPML (Rodén & Gedney 2000)**: same complex stretch, but the convolution is implemented through one auxiliary field per stretched derivative, evolved by a first-order ODE. Far fewer extra fields, more robust at grazing incidence, gracefully handles inhomogeneous media (like our lens). CPML is the modern standard.

In CPML, each stretched derivative $(1/s_x)\partial_x E$ is replaced by $\partial_x E + \Psi^x$ where the auxiliary Ψ^x evolves as

$$\frac{d\Psi^x}{dt} = -\alpha_x \Psi^x - \sigma_x \partial_x E.$$

Outside the PML, $\sigma_x = \alpha_x = 0$ and $\Psi^x \equiv 0$.

9.3 The damping profile and what a PML would add

$\sigma(x)$ starts at zero at the interface and rises smoothly to $\sigma_{\max}$ at the outer face: $\sigma(x) = \sigma_{\max} (\xi/L_{\text{PML}})^m$, $m = 3$ typical, with $\sigma_{\max}$ tuned so the round-trip reflection $R = \exp(-2\sigma_{\max}L_{\text{PML}}/((m+1)c))$ sits at the target ($R \sim 10^{-5}$). Adding CPML to `Maxwell_Kokkos` would require, roughly:

1. Pad the grid by $N_{\text{PML}} \approx 10$ cells per physical face.
2. Add six auxiliary Ψ fields per spatial derivative on the PML region.
3. Replace `SIMPLE_MAXWELL_INTERIOR_DOT` with a PML-aware cousin in the layer cells (a per-point helper function rather than a fixed-coefficient macro).
4. Apply matching damping profiles to the constraint-damping Ψ_D, Ψ_B blocks.
5. Use a trivial zero-Dirichlet (or zero-SAT) boundary at the outermost cell of the PML.

A few hundred lines of code and a week of careful convergence testing. Comparison summary:

	SBP-SAT (current)	PML (future)
Mechanism	Characteristic projection	Complex coordinate stretching
Normal incidence	Exact for $\tau \geq 1/2$	Near-exact, $R \sim 10^{-5}$
Grazing incidence	Strongly reflective	Near-exact
Extra grid cells	None	~ 10 per physical face
Extra fields	None	One Ψ per stretched derivative
Energy estimate	Yes (provable)	Indirect (round-trip bound)
Handles lens refl.	Head-on only	All angles
Code complexity	Low	Medium-high

The choice between SAT and PML is a design decision, not a correctness one. For a teaching code demonstrating SBP-SAT, SAT alone is appropriate. For production scattering simulations, PML is the right tool.

10 Software architecture: Kokkos integration

This section is the unique value-added of the Kokkos port. Sections 2–9 apply equally to the parent C version; the rest of this document is specific to `Maxwell_Kokkos`.

10.1 Driving design choice

A `Kokkos::View<double***, LayoutLeft>` sized (N_x, N_y, N_z) has the same flat-index offset $ijk = i + jN_x + kN_xN_y$ as the C version's `double*` arrays. Every kernel extracts a raw `double*` via `view.data()` *outside* the parallel region:

```
double *Bx      = gfs->evol[BX_SLOT].state.data();
double *dotBx   = gfs->evol[BX_SLOT].K[kidx].data();
// ...

Kokkos::parallel_for("interior", policy,
  KOKKOS_LAMBDA(int64_t i, int64_t j, int64_t k) {
    const int64_t ijk = i + j*nx + k*nx*ny;
    SIMPLE_MAXWELL_INTERIOR_DOT;    // uses Bx[ijk], dotBx[ijk], ...
  });
```

The lambda captures the pointers *by value* (`[=]`); on the OpenMP backend they point to host memory, on CUDA to device memory. Either way, `Bx[ijk]` resolves through the captured pointer to the right address.

The single payoff. The SymPy-generated `simple_maxwell.h` macros and the SBP-4-2 stencils in `derivatives.hpp` need no source changes other than annotating each `static inline` stencil with `KOKKOS_INLINE_FUNCTION`. The pointer-aliasing strategy keeps the access pattern `Bx[ijk]` working unchanged — so `generate_ccode.py` did not need updating, and `make check-generated` (the SymPy-vs-source consistency check from the parent) still passes byte-for-byte.

10.2 Three-region kernel partition

The time-derivative function `maxwell_eq_time_deriv` issues four `parallel_for` launches, mirroring the C version's three-region shell partition:

1. **Deep interior:** every cell sufficiently far from every physical face uses fixed D4CEN on all axes. No branches in the kernel body — this is the hot path.
2. **Shell A:** z -shell rows (k in the SBP closure zone), full (j, i) . Runtime stencil dispatch on all three axes via `stencil_at + apply_stencil`. Per-face SAT penalties added when the cell sits on a physical boundary row.
3. **Shell B:** y -shell rows, z in deep interior, full i . z -stencil known to be D4CEN; x, y dispatched at runtime.
4. **Shell C:** x -shell rows, y, z in deep interior. Only x dispatched at runtime.

The three shell regions partition the boundary shell exactly once, so each cell is visited at most twice (once interior, once shell) and each SAT face fires at most once per RK4 stage at any given cell, even at edges and corners. The same partition appears in `maxwell_constraints` for the diagnostic divergences c_D, c_B .

10.3 Ghost-zone exchange

`sync_vars(NGFS&, var_type, kidx)` performs the standard non-blocking neighbour exchange in three sequential axis sweeps (x , then y , then z). Pack and unpack are `parallel_for` kernels writing into a `Kokkos::View<double*>` per axis face; on the CUDA backend the device buffer is deep-copied to a host mirror before the MPI call (or sent directly when built with `MAXWELL_CUDA_AWARE_MPI=ON`).

The axis order is sequential rather than fully parallel because the y -axis send box includes x -ghost columns at $j \in [g_s, 2g_s)$; the y -sync only delivers correct corner data if x has already been synced. The Maxwell production stencils are axis-aligned and never read corner ghosts, so a fully parallel exchange would be functionally correct for the PDE solve, but the `test_sync` harness corrupts and verifies all ghost cells including corners. Sequential order matches the C version.

The special form `sync_vars(gfs, EVOLVED, -1)` syncs the `state` buffer instead of `K[kidx]` — used by the startup sync test in the driver to validate `set_initial_data`.

10.4 Reductions

`l2_error_analytic` is one `Kokkos::parallel_reduce` accumulating the squared L^2 residual across all nine evolved fields into a single `double`, followed by `MPI_Allreduce` on (sum,count). The collective is the only synchronisation between ranks during evaluation.

10.5 File map

File	Origin	Role
<code>driver.cpp</code>	ported	Top-level driver, RK4 loop
<code>maxwell_eqs.{hpp,cpp}</code>	ported	Time deriv + constraints + IC + L2
<code>rk4.{hpp,cpp}</code>	ported	Kokkos RK4
<code>numerical.{hpp,cpp}</code>	ported	<code>apply_dissipation</code>
<code>comm.{hpp,cpp}</code>	ported	Ghost-zone sync
<code>gf.{hpp,cpp}</code>	ported	View-owning NGFS
<code>io.{hpp,cpp}</code>	ported	HDF5 output + checkpoint/restart
<code>HDF5BinaryWrite.{hpp,cpp}</code>	ported	HDF5 file/group/dataset wrapper
<code>analytic_parameters.h</code>	ported	POD parameter structs (per-source)
<code>analytic_solutions.hpp</code>	ported	Kokkos-callable analytic evaluators
<code>derivatives.hpp</code>	ported	D4CEN, SBP-4-2 closures, dissipation
<code>simple_maxwell.h</code>	verbatim	SymPy-generated SAT & interior macros
<code>generate_ccode.py</code>	verbatim	SymPy emitter
<code>domain.{c,h}</code>	verbatim	MPI Cartesian topology
<code>maxwell_parameters.{h,cpp}</code>	verbatim	POD parameter ABI; TOML $\rightarrow$ <code>maxwell_param_st</code>
<code>parameter.{hpp,cpp}</code>	verbatim	Internal toml++ helpers (namespace <code>parameters</code>)
<code>toml.hpp</code>	verbatim	toml++ vendored header
<code>timer.{c,h}</code>	verbatim	Hierarchical timer
<code>tests/test_topology.c</code>	verbatim	MPI decomposition unit test
<code>tests/test_rk4.cpp</code>	ported	RK4 4th-order convergence test
<code>tests/test_sync.cpp</code>	ported	Ghost-zone exchange unit test

11 Build

11.1 Prerequisites

- CMake $\geq$ 3.18.
- C++20 compiler. Tested: GCC 13, GCC 15. GCC 12 has a known interaction with toml++; see §16.
- MPI (OpenMPI, MPICH, or any MPI-3 implementation).
- HDF5 with the C and HL components.
- For the GPU build: CUDA Toolkit $\geq$ 11.4 targeting an appropriate GPU architecture.

11.2 Default OpenMP build

```
cd Week15/Maxwell_Kokkos/src
cmake -S . -B build
cmake --build build -j
```

This produces `build/maxwell_system` plus the three test executables under `build/tests/`.

11.3 CUDA build

```
cmake -S . -B build \
  -DKokkos_ENABLE_OPENMP=OFF \
  -DKokkos_ENABLE_CUDA=ON \
  -DKokkos_ARCH_AMPERE80=ON
cmake --build build -j
```

Replace `Kokkos_ARCH_AMPERE80` with the `Kokkos_ARCH_*` variable for your GPU's compute capability (HOPPER90 for an H100, TURING75 for a T4, etc.).

11.4 CUDA-aware MPI

```
cmake -S . -B build \
  -DKokkos_ENABLE_OPENMP=OFF \
  -DKokkos_ENABLE_CUDA=ON \
  -DKokkos_ARCH_AMPERE80=ON \
  -DMAXWELL_CUDA_AWARE_MPI=ON
```

Skips the device–host buffer hop in `comm.cpp` and hands device pointers directly to MPI. Requires an MPI install built with CUDA awareness (e.g. OpenMPI `-with-cuda`).

11.5 CMake options reference

Option	Default	Effect
<code>Kokkos_ENABLE_OPENMP</code>	ON	OpenMP host backend
<code>Kokkos_ENABLE_CUDA</code>	OFF	CUDA device backend
<code>Kokkos_ARCH_<arch></code>	unset	Required when CUDA
<code>MAXWELL_CUDA_AWARE_MPI</code>	OFF	Device pointers to MPI
<code>CMAKE_BUILD_TYPE</code>	Release	Standard CMake
<code>CMAKE_CXX_STANDARD</code>	20	Hard requirement

11.6 Source-level switches

Defining `TIMING_ONLY` on the compile line skips HDF5 output and L^2 reporting in the main loop, leaving only kernel work for clean per-step timing.

12 Running

```
mpirun -np <N> ./maxwell_system path/to/maxwell.toml
```

12.1 Parameter file structure

[grid]

```
nx = 64          # global grid points in x (total, not per rank)
ny = 64
nz = 128
x0 = -1.0        # lower corner
y0 = -1.0
z0 = 0.0
xn = 1.0         # upper corner
yn = 1.0
zn = 4.0
periodic_x = false # if true: no SAT on this axis, ghost zones wrap
periodic_y = false
periodic_z = false
```

[solver]

```
ghost_size      = 3
cfl_factor      = 0.2  # <= 0.2 for all-physical; <= 0.5 for periodic x,y
max_iterations   = 2049
output_every    = 32
checkpoint_every = 512
max_checkpoints = 2
recover         = false
use_dissipation = false # Kreiss-Oliger; off by default
diss_coeff      = 0.1
tau            = 1.0   # SAT penalty strength, >= 1/2
```

[physics]

```
kappa_D = 0.1
kappa_B = 0.1
```

[source]

[source]

```
type = "gaussian_beam"  # or "plane_wave" or "te_waveguide_mode"
```

[source.plane_wave]

```
ax = 1.0      # x-polarisation amplitude
ay = 1.0      # y-polarisation amplitude
k  = 4.0      # wavenumber
bump_a = 0.0   # envelope turn-on start (in t-z)
bump_b = 2.0   # envelope turn-on end
```

[source.gaussian_beam]

```
w0 = 0.20     # waist radius
z_waist = 2.0  # z location of waist (inside domain for lens demo)
k  = 15.0     # wavenumber
amplitude = 1.0 # peak field amplitude
ramp_a = 0.0   # turn-on start (in t)
ramp_b = 1.0   # turn-on end
```

[source.te_waveguide_mode]

```
l = 2
m = 2
n = 2
```

```

[material]
[material]
epsilon_type = "elliptical"    # or "uniform"

[material.background]
epsilon = 1.0                  # baseline (asymptote of elliptical profile)
mu      = 1.0
sigma   = 0.0

[material.elliptical]
max = 4.0                     # peak permittivity at lens centre
x0  = 0.0
z0  = 2.0
s   = 2.0                     # steepness
a   = 0.5                     # semi-axis x
b   = 0.25                    # semi-axis z

```

12.2 Two canonical configurations

Option B: plane-wave convergence reference. Periodic x, y , non-periodic z , plane-wave source. The only configuration in which an exact, non-dispersive travelling-wave solution exists for the plane-wave source:

```

[grid]
periodic_x = true
periodic_y = true
periodic_z = false

[source]
type = "plane_wave"

```

On a sequence of refined grids, the L^2 error against the analytic reduces at the order of the SBP-(4,2) boundary closure (formally 2nd-order locally at the boundary, integrating to higher order globally; empirically around 3rd-order).

Option A: beam through lens. All six faces non-periodic with SAT, Gaussian-beam source, elliptical permittivity:

```

[grid]
periodic_x = false
periodic_y = false
periodic_z = false

[source]
type = "gaussian_beam"

[material]
epsilon_type = "elliptical"

```

12.3 Output layout

Each rank writes a self-describing `3D_rank_<R>.h5` containing the full state at every output interval, grouped by output index. Groups `/0, /1, /2, ...` correspond to the initial state, the first output step, etc. (step N is at time $N \cdot \text{output_every} \cdot dt$). Each group has datasets for the nine evolved variables

and the five auxiliaries (including the diagnostic divergences c_D, c_B). The `/metadata` group carries per-rank attributes describing the grid (origin, spacing, local and global sizes, ghost width, variable names) — everything needed to reassemble the global domain.

With the beam source’s smooth turn-on, group `/0` contains identically zero fields. The first non-trivial output is at $t = \text{output_every} \cdot dt$; by $t \geq \text{ramp_b}$ the envelope has saturated.

An L^2 diagnostic `l2_norm.dat` is written by rank 0 at every output step, containing $(t, \|U_{\text{numerical}} - U_{\text{analytic}}\|_{L^2})$. For `plane_wave` with periodic x, y and for `te_waveguide_mode`, this is a real convergence metric. For `gaussian_beam` the analytic reference is only the paraxial approximation, so the L^2 error reaches a paraxial-scale plateau, not zero.

12.4 Visualization

```
python scripts/make_xdmf.py --dir <run_dir> --dt-per-output T
```

where $T = \text{output_every} \cdot c \cdot \min(dx, dy, dz)$ is the physical time between output groups. The script scans the per-rank HDF5 files, reads their `/metadata`, and emits a single `maxwell.xmf` alongside. The manifest is pure XML metadata — the payload bytes stay in HDF5. The result opens directly in:

- ParaView (Xdmf3 reader preferred),
- VisIt (Xdmf plugin),
- PyVista (`pyvista.XdmfReader("maxwell.xmf")`).

A time slider stepping through the N output steps appears automatically; all 14 fields are selectable as point-data attributes.

Grid-layout details. The XDMF uses a `3DRectMesh` topology with explicit `VXVYVZ` geometry (one per-axis coordinate list). This avoids the long-standing XDMF `ORIGIN_DXDYDZ` ambiguity in which VisIt and ParaView disagree on whether the origin tuple is in spatial (x, y, z) or topology (z, y, x) order; `VXVYVZ` tags each axis explicitly.

Legacy fallback. `scripts/hdf5_to_vtk.py` reassembles the global grid on rank 0 and writes one VTI per time step. Serial, duplicates the data on disk; supplied only as a fallback.

12.5 Checkpoint restart

Set `[solver].recover = true` and re-run with the same TOML. The driver locates the highest-iteration checkpoint for this rank and resumes; HDF5 group numbering continues monotonically from the recovered iteration.

12.6 Diagnostics and common problems

L^2 error blows up exponentially. Usually CFL is too large. Reduce `cfl_factor` to 0.2 (six-face SAT) or verify the configuration is in fact Option B if you expected `cfl = 0.4` to work.

D field looks empty in visualization. Check the time slice. Group `/0` is the initial state, which is zero for both plane-wave and Gaussian-beam sources. For the default ramp `[0, 1]` on the shipped TOML, the beam reaches $\sim 3\%$ by group `/1` ($t = 0.2$) and full amplitude by $t \geq \text{ramp_b}$; visualise group `/5` or later for the default parameters.

Paraxial residual looks larger than expected. Verify $k \cdot w_0 \gtrsim 5$. Residual scales as $1/(kw_0)^2$.

Beam appears to reflect off the transverse walls. The x, y faces use $g = 0$ SAT; reflection amplitude is proportional to the field amplitude at the wall. Increase L_x, L_y or reduce w_0 so the tail is negligible. Rule of thumb: $L_x \geq 10 w_0$, where w_0 is the local spot size along the propagation, not the waist.

High-frequency oblique wavefronts at late times. Combined signature of (a) a real standing wave from incident + lens-reflected beam interfering every $\lambda/2 = \pi/k$ and (b) spurious angular reflections of lens-scattered energy off every SAT face. (a) is physics; (b) is the SAT limitation (§8.6). Enabling Kreiss-Oliger dissipation suppresses the high-frequency component. For truly angle-independent absorption the code would need a PML (§9).

13 Tests

Three test executables ship with the build, plus a Python convergence harness (§14).

13.1 test_topology

Pure CPU; exercises `automatic_topology`, the greedy prime-factor MPI Cartesian decomposition. Single-process; runs through a battery of grid shapes and process counts.

13.2 test_rk4

Solves $dy/dt = -y$ with $y(0) = 1$ on a small grid for $t \in [0, 1]$ at two time-step sizes. Asserts the error ratio between $\Delta t = 0.01$ and $\Delta t/2$ is between 12 and 20, i.e. within tolerance of the expected fourth-order rate $2^4 = 16$. Drives the Kokkos RK4 + sync paths end-to-end on a trivial PDE.

13.3 test_sync

The infrastructure correctness test. Workflow: fill all evolved and auxiliary fields with a known smooth function on the local grid, corrupt every ghost cell with a sentinel value, call `sync_vars`, then verify every cell (including ghosts) matches the analytic value. Driven by `run_tests.sh` at 1, 2, 4, 6, 8, and 27 ranks across three boundary configurations: fully physical (0 0 0), periodic x, y + physical z (1 1 0, the Maxwell default), and fully periodic (1 1 1).

13.4 Running the suite

```
cd build
ctest                # runs tests/run_tests.sh
                     # = test_topology, test_rk4, and the
                     # 12-config test_sync grid

cd ..
python scripts/convergence_test.py \
    --solver src/build/maxwell_system \
    --resolutions 16 24 32 48 \
    --final-time 0.1 --np 4
```

14 Verification: convergence testing

A direct test for the correctness of the interior stencil, each face’s SBP-SAT closure, and the additive penalty at edges and corners is obtained by forcing the numerical solution to converge to a known analytic solution and measuring the convergence rate.

14.1 Test problem

The test uses the `te_waveguide_mode` (§6.4), which is an exact vacuum Maxwell solution with $\varepsilon = \mu = 1, \sigma = 0$ on the unit cube $[0, 1]^3$. When `[source].type = "te_waveguide_mode"`, the code routes the analytic as boundary data g on *every* physical face (not just lower- z). At each SAT-face point, the scheme evaluates the analytic at the local (x, y, z, t) and passes it into the relevant `APPLY_SAT_*` macro. The numerical state is driven toward the analytic through the incoming-characteristic projection of every physical face; the outgoing characteristics are untouched, so the test is consistent with the method’s energy argument.

The scheme being tested is:

- Deep interior: 4th-order centered stencil `D4CEN` in all three directions.
- Near-boundary rows on physical axes: SBP 4-2 one-sided closures.
- Exactly one boundary row per physical face: SAT penalty with g from the analytic.
- Edges (two faces meet): two SAT penalties additive.
- Corners (three faces meet): three SAT penalties additive.
- Time integration: classical RK4.

14.2 The four configurations

The boundary periodicities are varied per axis to change which boundary structures are exercised:

Configuration	<code>periodic_x</code>	<code>periodic_y</code>	<code>periodic_z</code>	Faces / Edges / Corners	Expected L^2 rate
<code>fully_periodic</code>	true	true	true	0 / 0 / 0	4
<code>z_physical</code>	true	true	false	2 / 0 / 0	3
<code>yz_physical</code>	true	false	false	4 / 4 / 0	3
<code>all_physical</code>	false	false	false	6 / 12 / 8	3

What each configuration localises.

- `fully_periodic`: no SBP closures, no SAT. All derivatives are `D4CEN` with periodic ghosts. Tests only the interior stencil. Expected rate 4.
- `z_physical`: only the two z -faces are physical; x, y stay periodic. Tests interior + z -face SBP + SAT. No physical edges or corners. A drop here means face-level SBP or SAT is broken.
- `yz_physical`: adds four y - z edges. A drop from `z_physical` to `yz_physical` means the additive edge penalty is broken.
- `all_physical`: full setup — six faces, twelve edges, eight corners. A drop here means the three-face corner penalty is broken.

14.3 Why the expected rates are 4 and 3

The scheme is formally:

- 4th-order in the interior (D4CEN is $O(h^4)$).
- 2nd-order locally at the SBP-4-2 boundary rows (Strand closures are $O(h^2)$).

For a hyperbolic system discretized by an SBP operator with interior order $2p$ and boundary truncation order s , the classical Svård–Nordström result predicts a global L^2 rate of $\min(2p, s + 1)$ at late times. For SBP-4-2 with $2p = 4, s = 2$: rate 3. The interior-only configuration sidesteps the boundary closures and retains the full 4th-order rate.

Two practical notes:

- The $s + 1$ rule applies per face; edges and corners do not further degrade the rate because the additive SAT gives independent energy estimates that combine as a maximum, not a product. This is the feature being tested.
- Measured rates often approach 3 from above at finite N — rates of 3.1–3.2 at $N = 48$ are typical for the SBP-4-2 scheme; they asymptote to exactly 3 as $N \rightarrow \infty$. A rate persistently *below* 3 at moderate N indicates a real problem.

14.4 Running the convergence test

```
python scripts/convergence_test.py \
--solver src/build/maxwell_system \
--resolutions 16 24 32 48 \
--final-time 0.1 \
--np 4
```

Options: `-solver` PATH, `-resolutions` N1 N2 ..., `-final-time` T, `-cfl` C (default 0.2), `-np` N, `-work-dir` DIR.

The driver runs the four configurations in sequence; each prints per-resolution errors and pairwise rates. A summary table at the end compares measured final rates against expected:

=== SUMMARY ===

config	N=16	N=24	N=32	N=48	final rate	
fully_periodic	0.001041	0.0002078	6.597e-05	1.306e-05	3.994	OK
z_physical	0.00739	0.002273	0.0009391	0.000261	3.158	OK
yz_physical	0.009489	0.002867	0.001174	0.000325	3.167	OK
all_physical	0.01092	0.003261	0.001328	0.000368	3.166	OK

A rate is flagged **SUSPICIOUS** if it deviates from expected by more than 1.0; the driver exits nonzero.

14.5 Baseline results (Kokkos OpenMP backend, validated)

Configuration	$N = 16$	$N = 24$	$N = 32$	$N = 48$	Final rate
fully_periodic	$1.04 \cdot 10^{-3}$	$2.08 \cdot 10^{-4}$	$6.60 \cdot 10^{-5}$	$1.31 \cdot 10^{-5}$	3.99
z_physical	$7.39 \cdot 10^{-3}$	$2.27 \cdot 10^{-3}$	$9.39 \cdot 10^{-4}$	$2.61 \cdot 10^{-4}$	3.16
yz_physical	$9.49 \cdot 10^{-3}$	$2.87 \cdot 10^{-3}$	$1.17 \cdot 10^{-3}$	$3.25 \cdot 10^{-4}$	3.17
all_physical	$1.09 \cdot 10^{-2}$	$3.26 \cdot 10^{-3}$	$1.33 \cdot 10^{-3}$	$3.68 \cdot 10^{-4}$	3.17

These match the C-version’s reference numbers from `Maxwell_Penalty/Review.md §1` to four significant figures. Run parameters: $T_{\text{final}} = 0.1$, $\text{cfl} = 0.2$, $\tau = 1.0$, four MPI ranks, uniform $\varepsilon = \mu = 1$, $\sigma = 0$, mode $(l, m, n) = (2, 2, 2)$.

14.6 Interpreting a failed test

If a future change causes a rate to drop, the configuration hierarchy localises the fault:

- `fully_periodic` drops below 4: the interior D4CEN stencil, ghost-zone sync, or periodic wrap is broken.
- `z_physical` drops below 3 while `fully_periodic` stays at 4: the z -face SBP closures or the lower/upper- z SAT penalty is broken.
- `yz_physical` drops below 3 while `z_physical` is fine: the additive edge penalty at y - z edges is broken.
- `all_physical` drops below 3 while `yz_physical` is fine: the three-face corner penalty is broken.

A successful pass on all four rows confirms that every part of the algorithm — interior stencil, face closure, face SAT, edge additivity, corner additivity — is correctly implemented.

A subtle point: why the oblique-reflection failure mode of §8.6 does not break this test.

The `te_waveguide_mode` has nontrivial structure in every direction and hits the $\pm x, \pm y$ faces at oblique angles — the exact configuration mis-handled by the $g = 0$ SAT used in the lens demo. One could reasonably expect spurious reflections to pollute the rate.

This does not happen because the convergence test uses $g = \text{analytic}$ on every physical face, not $g = 0$. The absorbing-boundary failure mode arises only when the prescribed incoming data disagrees with the true physical solution at the boundary: with $g = 0$ and a nonzero tangential field, the SAT pulls the wave toward zero instead of toward its true value, generating the spurious reflection. When g equals the exact analytic at every boundary point, the SAT imposes the correct incoming data on every face regardless of incidence angle. Mathematically, the SAT penalty is a weak-form Dirichlet imposition of the incoming characteristic; with the correct right-hand side it has no angular blind spot.

The convergence test therefore verifies that the SBP-SAT scheme, *given the correct incoming data*, converges to the analytic at the expected rate. It is a test of the scheme’s implementation, not of the quality of absorbing boundaries in the lens-demo sense.

Caveats.

- The `te_waveguide_mode` source requires $\varepsilon = \mu = 1, \sigma = 0$. The test does not exercise spatially-varying permittivity; that’s a separate physics question.
- The constraint fields Ψ_D, Ψ_B are identically zero in the analytic. The constraint-damping blocks of the SAT are exercised only by their coupling to the EM blocks through residual numerical violations, so a bug localised entirely to those blocks might not be caught at full sensitivity.
- The test uses $g = \text{analytic}$ on every face and therefore does *not* measure the quality of the $g = 0$ absorbing boundary on oblique waves — that is a physics question (§8.6) and a design question about replacing SAT with PML (§9), not an implementation question about the SAT itself.

15 Validation status

15.1 Verified on the OpenMP backend

- All 14 `run_tests.sh` configurations pass: `test_topology`, `test_rk4`, and 12 `test_sync` cases at 1–27 ranks across three boundary configurations.
- Convergence test passes at all four configurations to within 5% of the expected asymptote at $N = 16, 24, 32, 48$: 3.99 (fully periodic), 3.16/3.17/3.17 (boundary configurations) at $N = 48$. These rates match the C-version’s published reference numbers in `Maxwell_Penalty/Review.md` §1 to four significant figures.
- Smoke runs of both demo TOMLs (`maxwell.toml`, `maxwell_waveguide.toml`) complete normally with the documented L^2 behaviour.

15.2 Built but not yet runtime-verified

- **CUDA backend.** Builds cleanly on an A100 host with the toolchain pitfalls in §16 resolved. Runtime convergence test on the GPU is the recommended next validation step.
- **CUDA-aware MPI.** The `MAXWELL_CUDA_AWARE_MPI=ON` compile path compiles, but no test distinguishes a CUDA-aware build from the default host-mirror build other than performance.
- **Checkpoint restart.** Both `write_checkpoint` and `read_checkpoint` are ported and link, but no automated round-trip verification is in the test suite.
- **make check-generated.** `generate_ccode.py` is unchanged from the C version and emits a `simple_maxwell.h` byte-for-byte identical to the checked-in copy, but no CMake target exercises this.

15.3 Recommended GPU-side test sequence

After a successful CUDA build, validate in this order:

1. `./tests/test_rk4` — expect Kokkos exec space: Cuda and the 4th-order ratio check to pass.
2. `./tests/test_sync 32 32 32 0 0 0 at -np 1` — exercises the device pack/unpack and the host-mirror MPI hop.
3. `python scripts/convergence_test.py -np 1 -resolutions 16 24 32 -final-time 0.1` — the strongest single check. Each rate should match the OpenMP baseline to better than 0.5%.
4. Diff a 50-step `maxwell.toml` run’s `l2_norm.dat` between OpenMP and CUDA — should agree to 10^{-12} or better at every output step.

16 Known issues and toolchain workarounds

16.1 OpenMPI 4.x C++ bindings vs. nvcc

OpenMPI’s `mpi.h` pulls in `mpi/cxx/functions.h` and `mpi/cxx/file.h`, headers from the deprecated MPI-2 C++ namespace. Those headers declare member functions like `MPI::Init()` inside an `extern "C"` block with overloads — legal under `g++` (which silently picks one), illegal under `nvcc`’s stricter linkage check.

Fix (already in `CMakeLists.txt`):

```
add_compile_definitions(OMPI_SKIP_MPICXX MPICH_SKIP_MPICXX)
```

Both macros are recognised by their respective MPI implementations and skip the entire `mpi/cxx/*.h` tree. The code uses only the C MPI API, so dropping the C++ bindings has no functional effect.

16.2 GCC 12 + toml++ + C++20

GCC 12 (and only GCC 12) miscompiles a generic-lambda `if constexpr` pattern used by toml++:

```
using return_type = decltype(visitor(...));
if constexpr (impl::is_constructible_or_convertible<bool, return_type>)
```

The compiler mangles `return_type` as `__T<N>` and then fails to find the name in the `if constexpr` expression.

Fix (design-level). The original parameter header served two unrelated roles: declaring the maxwell-specific POD parameter ABI (structs + `parse_maxwell_parameters()`) and providing toml++-typed helpers (`namespace parameters { ... }`) used internally by the parser implementation. These were split into two headers:

- `maxwell_parameters.h` — public ABI: the POD structs the rest of the solver consumes plus the C-callable `parse_maxwell_parameters()`. No toml++. Included by `driver.cpp` (transitively via `maxwell_eqs.hpp`) and any consumer of the parsed result.
- `parameter.hpp` — internal header of the parser: `namespace parameters` helpers built on `toml::table`. Pulls `toml.hpp`. Included only by `parameter.cpp` (which implements the namespace) and `maxwell_parameters.cpp` (which uses it to populate the structs).

toml++ exposure is now implicit in including `parameter.hpp`; nothing else needs it. Translation units that never include `parameter.hpp` never see the buggy toml++ lambda. Side benefit: a 15,000-line header is no longer pulled into the kernel-heavy translation units, which substantially shortens compile time.

If a future patch invokes the toml++ helpers from a new translation unit and the GCC 12 bug resurfaces, the in-place workaround in `toml.hpp` is to bind the trait to a local `constexpr bool`:

```
using return_type = decltype(...);
constexpr bool _returns_bool =
    impl::is_constructible_or_convertible<bool, return_type>;
if constexpr (_returns_bool) { ... }
```

Apply at both `toml.hpp:8239` and `:8258`.

16.3 Single-GPU multi-rank

By default each MPI rank tries to bind device 0. CUDA contexts can be shared, so this is correct, but performance is degraded and some operations may serialise. Workarounds:

- `-np 1` for single-GPU correctness checks.
- NVIDIA Multi-Process Service (`nvidia-cuda-mps-control -d`) for ≤ 2 –4 ranks.
- On multi-GPU nodes, one rank per GPU (`mpirun -map-by ppr:1:gpu`).

16.4 C++20 hard requirement

Kokkos 5.x rejects C++17 outright. The C version’s TOML files build under C++17, but the Kokkos build needs at least C++20. GCC 13+, Clang 16+, NVHPC 24+ all satisfy this.

References

- Strand, B. (1994), *Summation by parts for finite difference approximations of d/dx* , J. Comput. Phys. 110, 47–67. Source for the SBP-4-2 norm coefficients and one-sided closures used in `derivatives.hpp`.
- Carpenter, M., Gottlieb, D., Abarbanel, S. (1994), *Time-stable boundary conditions for finite-difference schemes solving hyperbolic systems: methodology and application to high-order compact schemes*, J. Comput. Phys. 111, 220–236. Original SAT derivation.
- Svärd, M., Nordström, J. (2014), *Review of summation-by-parts schemes for initial-boundary-value problems*, J. Comput. Phys. 268, 17–38. Source of the $\min(2p, s+1)$ global L^2 rate estimate used in §14.
- Munz, C.-D., Omnes, P., Schneider, R., Sonnendrücker, E., Voss, U. (2000), *Divergence correction techniques for Maxwell solvers based on a hyperbolic model*, J. Comput. Phys. 161, 484–511. The constraint-damping extension used here.
- Bérenger, J.-P. (1994), *A perfectly matched layer for the absorption of electromagnetic waves*, J. Comput. Phys. 114, 185–200. Original split-field PML.
- Roden, J. A., Gedney, S. D. (2000), *Convolutional PML (CPML): an efficient FDTD implementation of the CFS-PML for arbitrary media*, Microwave Opt. Technol. Lett. 27, 334–339. The CPML variant recommended for the proposed extension in §9.
- Kokkos team (2024). *Kokkos C++ Performance Portability Programming EcoSystem*, <https://kokkos.org>.
- Trott, C. et al. (2022), *Kokkos 3: Programming Model Extensions for the Exascale Era*, IEEE TPDS.

A Summary of SAT formulas

For reference, the full set of SAT contributions applied at the boundary rows, derived in §5. Let $s = \tau \cdot (H_0^{-1})/h$ be the per-axis penalty scale (with $H_0^{-1} = 48/17$ for the (4, 2) SBP norm), and define c, PP as in §5.2. Where g is zero, the δ quantities reduce to the evolved fields themselves.

Lower- z (nonzero g , the incoming beam):

$$\begin{aligned}
 \dot{B}_x &= s \cdot \frac{c}{2} (\delta B_x - PP \cdot \delta D_y) \\
 \dot{D}_y &= s \cdot \frac{c}{2} (-\delta B_x / PP + \delta D_y) \\
 \dot{B}_y &= s \cdot \frac{c}{2} (\delta B_y + PP \cdot \delta D_x) \\
 \dot{D}_x &= s \cdot \frac{c}{2} (\delta B_y / PP + \delta D_x) \\
 \dot{B}_z &= s \cdot \frac{1}{2} (\delta B_z - \delta \Psi_B) \\
 \dot{\Psi}_B &= s \cdot \frac{1}{2} (-\delta B_z + \delta \Psi_B) \\
 \dot{D}_z &= s \cdot \frac{1}{2} (\delta D_z - \delta \Psi_D) \\
 \dot{\Psi}_D &= s \cdot \frac{1}{2} (-\delta D_z + \delta \Psi_D)
 \end{aligned}$$

Upper- z ($g = 0$):

$$\begin{aligned}
\dot{B}_x &= s \cdot \frac{c}{2}(B_x + PP \cdot D_y) \\
\dot{D}_y &= s \cdot \frac{c}{2}(B_x/PP + D_y) \\
\dot{B}_y &= s \cdot \frac{c}{2}(B_y - PP \cdot D_x) \\
\dot{D}_x &= s \cdot \frac{c}{2}(-B_y/PP + D_x) \\
\dot{B}_z &= s \cdot \frac{1}{2}(B_z + \Psi_B) \\
\dot{\Psi}_B &= s \cdot \frac{1}{2}(B_z + \Psi_B) \\
\dot{D}_z &= s \cdot \frac{1}{2}(D_z + \Psi_D) \\
\dot{\Psi}_D &= s \cdot \frac{1}{2}(D_z + \Psi_D)
\end{aligned}$$

Lower- x , lower- y , upper- x , upper- y : obtained from the lower- z and upper- z formulas above by cyclic permutation of the axes. The explicit forms live in `simple_maxwell.h`, macros `APPLY_SAT_{LOWER,UPPER}_{X,Y,Z}`